Circles-lbp-group Audit

Côme du Crest

2025-16-05

Table of contents

- Table of contents
- Cicles-lbp-group Audit
  - Scope
  - Context
  - Status
- Issues
  - [Low] Tokens are not scaled to 18 decimals when validating amounts

Cicles-lbp-group Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes contracts within `aboutcircles/circles-lbp/src/base-group` as of commit `0xd5a52e1`.

Context

The goal of these contracts is to facilitate creation of a balancer pool on a group token and another user-specified asset.

Status

The report has been sent to the core developer.

Issues

[Low] Tokens are not scaled to 18 decimals when validating amounts

Summary

When starting the creation of an LBP, the factory validate the used token amounts and weights to ensure the balancer pool will not be created with a too low amount of LP tokens causing a revert in the balancer code for initializing a pool.

However, the tokens amount used to compute the invariant are not scaled to 18 decimals like it is done in the balancer code so the comparison with `MIN_BPT = 1e6` may be incoherent.

Vulnerability Detail

The function `BasePool.onJoinPool()` calls `_onInitializePool()` with the computed scaling factors before verifying that the amount of BPT out is greater than the minimal amount:

```
1     function onJoinPool(
2         bytes32 poolId,
3         address sender,
4         address recipient,
5         uint256[] memory balances,
6         uint256 lastChangeBlock,
7         uint256 protocolSwapFeePercentage,
8         bytes memory userData
9     ) external override onlyVault(poolId) returns (uint256[] memory, uint256[]
          memory) {
10        _beforeSwapJoinExit();
11
12        uint256[] memory scalingFactors = _scalingFactors();
13
14        if (totalSupply() == 0) {
15            (uint256 bptAmountOut, uint256[] memory amountsIn) =
                  _onInitializePool(
16                poolId,
17                sender,
18                recipient,
19                scalingFactors,
20                userData
21            );
22
23            ...
24            _require(bptAmountOut >= _getMinimumBpt(), Errors.MINIMUM_BPT);
25            ...
26        } ...
27    }
```

`BaseWeightedPool._onInitializePool()` upscales the amount of input tokens before computing the invariant and returning the corresponding amount of BPT out:

```
    function _onInitializePool(
        bytes32,
        address,
        address,
        uint256[] memory scalingFactors,
        bytes memory userData
    ) internal virtual override returns (uint256, uint256[] memory) {
        ...
        uint256[] memory amountsIn = userData.initialAmountsIn();
        ...
        _upscaleArray(amountsIn, scalingFactors);

        uint256[] memory normalizedWeights = _getNormalizedWeights();
        uint256 invariantAfterJoin = WeightedMath._calculateInvariant(
            normalizedWeights, amountsIn);
        ...
    }
```

The scaling function uses scaling factors to bring the amounts to 18 decimals:

```
    function _computeScalingFactor(IERC20 token) internal view returns (uint256
        ) {
        if (address(token) == address(this)) {
            return FixedPoint.ONE;
        }

        // Tokens that don't implement the `decimals` method are not supported.
        uint256 tokenDecimals = ERC20(address(token)).decimals();

        // Tokens with more than 18 decimals are not supported.
        uint256 decimalsDifference = Math.sub(18, tokenDecimals);
        return FixedPoint.ONE * 10**decimalsDifference;
    }
```

The minimum BPT comparison in `BasePool.onJoinPool()` uses `1e6` as the minimum amount of BPT:

```
    uint256 private constant _DEFAULT_MINIMUM_BPT = 1e6;

    function _getMinimumBpt() internal pure virtual returns (uint256) {
        return _DEFAULT_MINIMUM_BPT;
    }
```

The function `GroupLBPFactory._validateAmounts()` indeed uses the same value of `1e6` but it does not scale the amount of tokens to 18 decimals before computing the invariant so the comparison is not coherent with balancer's code.

```
    uint256 private constant MIN_BPT = 1e6;

    function _validateAmounts(bool crcZero, uint256 groupInitWeight, uint256
```

```
            groupAmount, uint256 assetAmount)
4           internal
5           view
6       {
7           if (groupAmount == 0 || assetAmount == 0) revert InvalidAmount();
8           uint256[] memory normalizedWeights = new uint256[](2);
9           normalizedWeights[0] = crcZero ? groupInitWeight : 1 ether -
                groupInitWeight;
10          normalizedWeights[1] = crcZero ? 1 ether - groupInitWeight :
                groupInitWeight;
11
12          uint256 stableCRCAmount = getStableAmount(groupAmount);
13
14          uint256[] memory amounts = new uint256[](2);
15          amounts[0] = crcZero ? stableCRCAmount : assetAmount;
16          amounts[1] = crcZero ? assetAmount : stableCRCAmount;
17
18          uint256 bptAmount = BALANCER_WEIGHTED_MATH.calculateInvariant(
                normalizedWeights, amounts) * 2;
19          if (bptAmount < MIN_BPT) revert InvalidAmount();
20      }
```

Impact

If tokens used as assets in `GroupLBPFactory` do not have 18 decimals, the function `_validateAmounts()` may accept too low amounts it should reject to ensure the pool can be created.

Code Snippets

https://github.com/aboutcircles/circles-lbp/blob/d5a52e1681d01770afa09ae19f8cfa1ecce009f4/src/base-group/GroupLBPFactory.sol#L395-L412

https://github.com/balancer/balancer-v2-monorepo/blob/36d282374b457dddea828be7884ee0d185db06ba/pkg/pool-utils/contracts/BasePool.sol#L289

https://github.com/balancer/balancer-v2-monorepo/blob/36d282374b457dddea828be7884ee0d185db06ba/pkg/pool-weighted/contracts/BaseWeightedPool.sol#L175

https://github.com/balancer/balancer-v2-monorepo/blob/36d282374b457dddea828be7884ee0d185db06ba/pkg/pool-utils/contracts/BasePool.sol#L615

Recommendation

Scale the amount of tokens to 18 decimals.